

Sztuczna Inteligencja

Projekt zaliczeniowy

Sieć głęboka

Wielowarstwowy perceptron do klasyfikacji
defektów blach stalowych

Autor:

Michał Cwynar

Nr albumu: 179465

Grupa: L09

12 maja 2026

Spis treści

1	Opis teoretyczny	4
1.1	Uczenie głębokie – definicja, geneza i znaczenie	4
1.2	Matematyczne podstawy – sformułowanie problemu uczenia	4
1.3	Sztuczny neuron – model matematyczny	5
1.3.1	Krok 1 – suma ważona (preaktywacja)	5
1.3.2	Krok 2 – funkcja aktywacji	5
1.4	Architektura wielowarstwowej sieci neuronowej (MLP)	6
1.4.1	Obliczenia w warstwie – notacja macierzowa	6
1.5	Funkcje aktywacji	6
1.5.1	ReLU (Rectified Linear Unit)	7
1.5.2	Sigmoid	7
1.5.3	Softmax – warstwa wyjściowa dla klasyfikacji wieloklasowej	7
1.6	Funkcja straty	8
1.6.1	Kategoryczna entropia krzyżowa	8
1.6.2	Związek z maksymalizacją wiarygodności	8
1.7	Propagacja w przód	8
1.8	Propagacja wsteczna – algorytm backpropagation	9
1.8.1	Reguła łańcuchowa	9
1.8.2	Sygnał błędu warstwy wyjściowej	9
1.8.3	Propagacja sygnału błędu przez warstwy ukryte	9
1.8.4	Gradienty parametrów	9
1.8.5	Pełny algorytm backpropagation	10
1.9	Optymalizacja parametrów – metody gradientowe	10
1.9.1	Podstawowy spadek gradientowy	10
1.9.2	Problem lokalnych minimów i obszarów siodłowych	10
1.9.3	Algorytm Adam (Adaptive Moment Estimation)	10
1.10	Cykl uczenia – epoki, mini-batche i preprocessing	11
1.10.1	Preprocessing – standaryzacja danych	11
1.10.2	Podział danych i pojęcie epoki	11
1.10.3	Ocena modelu – metryki	12
1.11	Inicjalizacja wag i regularyzacja	12
1.11.1	Inicjalizacja Kaiminga (He initialization)	12
1.11.2	Regularyzacja – zapobieganie przetrenowaniu	12

1.12	Podsumowanie – pełny cykl uczenia głębokiego	12
2	Opis projektu	14
2.1	Opis zbioru danych	14
2.1.1	Struktura danych wejściowych	14
2.1.2	Zmienne docelowe	15
2.2	Opis implementacji	16
2.2.1	Import bibliotek i konfiguracja akceleracji sprzętowej	16
2.2.2	Pobranie danych i rozdzielenie cech od etykiet	17
2.2.3	Konwersja etykiet z formatu one-hot na indeksy klas	17
2.2.4	Podział danych na zbiór treningowy i testowy	17
2.2.5	Standaryzacja danych wejściowych	17
2.2.6	Konwersja danych na tensory PyTorch	17
2.2.7	Utworzenie iteratorów danych (DataLoader)	18
2.2.8	Definicja architektury sieci neuronowej	18
2.2.9	Konfiguracja funkcji straty i optymalizatora	19
2.2.10	Pętla treningowa modelu	19
2.2.11	Ewaluacja końcowa na zbiorze testowym	20
3	Eksperymenty	21
3.1	Wpływ współczynnika uczenia	21
3.2	Porównanie funkcji aktywacji	22
3.3	Wpływ liczby neuronów w pierwszej warstwie ukrytej (K_1)	23
3.4	Wpływ liczby neuronów w drugiej warstwie ukrytej (K_2)	24
3.5	Wpływ liczby warstw ukrytych	25
3.6	Wspólny wpływ liczby neuronów K_1 i K_2	26
4	Podsumowanie	28
	Literatura	29
	Spis rysunków	30
	Spis listingów	31

1. Opis teoretyczny

1.1. Uczenie głębokie – definicja, geneza i znaczenie

Uczenie głębokie (ang. *deep learning*) jest poddziedziną uczenia maszynowego (ang. *machine learning*), które z kolei stanowi gałąź sztucznej inteligencji. Modele uczenia głębokiego reprezentowane są przez wielowarstwowe sztuczne sieci neuronowe, zdolne do automatycznego wyuczenia hierarchicznych, nieliniowych reprezentacji danych bezpośrednio z surowych danych wejściowych, bez potrzeby ręcznego projektowania cech (ang. *feature engineering*) – co odróżnia je od klasycznych algorytmów uczenia maszynowego [1].

Przymiotnik „głębokie” odnosi się do liczby warstw ukrytych sieci. Im więcej warstw, tym sieć jest w stanie modelować coraz bardziej abstrakcyjne zależności. Dzięki kompozycji wielu prostych przekształceń nieliniowych możliwe jest budowanie reprezentacji wielopoziomowych: od krawędzi i tekstur w warstwach wejściowych po semantyczne koncepcje w warstwach głębokich.

Historia uczenia głębokiego sięga lat 40. XX wieku. W 1943 roku Warren McCulloch i Walter Pitts zaproponowali pierwszy matematyczny model neuronu. W 1958 roku Frank Rosenblatt skonstruował perceptron – prosty klasyfikator liniowy. Kluczowym przełomem był rok 1986, kiedy Rumelhart, Hinton i Williams opublikowali pracę opisującą algorytm propagacji wstecznej [2]. Prawdziwy rozkwit nastąpił po 2010 roku, dzięki dostępności dużych zbiorów danych, procesorów graficznych (GPU) oraz nowych funkcji aktywacji. Sieć AlexNet [3] zredukowała błąd klasyfikacji ImageNet o ponad 10 punktów procentowych względem ówczesnych metod klasycznych.

1.2. Matematyczne podstawy – sformułowanie problemu uczenia

Uczenie sieci głębokiej polega na znalezieniu takich wartości parametrów θ (macierzy wag $\mathbf{W}^l$ i wektorów biasów $\mathbf{b}^l$ dla każdej warstwy l), aby model f jak najlepiej odwzorowywał zależności między wejściami $\mathbf{x}$ a oczekiwanymi wyjściami $\mathbf{y}$. Formalnie jest to zadanie minimalizacji oczekiwanej wartości funkcji straty $\mathcal{L}$ (ang. *expected risk minimization*) [1]:

$$\theta^* = \arg \min_{\theta} \mathbb{E}_{(\mathbf{x}, \mathbf{y}) \sim p_{\text{data}}} [\mathcal{L}(f(\mathbf{x}; \theta), \mathbf{y})], \quad (1.1)$$

gdzie $f(\mathbf{x}; \theta)$ jest funkcją realizowaną przez całą sieć, $\mathcal{L}$ mierzy rozbieżność między

predykcją a etykietą, a wartość oczekiwana liczona jest względem prawdziwego rozkładu danych p_{data} . Ponieważ rozkład p_{data} nie jest znany, zastępuje się go empiryczną aproksymacją na skończonym zbiorze treningowym $\mathcal{D} = \{(\mathbf{x}_i, \mathbf{y}_i)\}_{i=1}^N$. Otrzymujemy *empiryczne ryzyko*:

$$\theta^* \approx \arg \min_{\theta} \frac{1}{N} \sum_{i=1}^N \mathcal{L}(f(\mathbf{x}_i; \theta), \mathbf{y}_i). \quad (1.2)$$

Jest to zasada *minimalizacji empirycznego ryzyka* (ang. *empirical risk minimization, ERM*). Uczenie realizowane jest iteracyjnie – w każdym kroku model wykonuje propagację w przód, oblicza stratę, wylicza gradienty metodą propagacji wstecznej i aktualizuje parametry.

1.3. Sztuczny neuron – model matematyczny

Podstawową jednostką obliczeniową sieci głębokiej jest sztuczny neuron, będący matematycznym modelem neuronu biologicznego. Neuron o n wejściach oblicza wynik w dwóch krokach.

1.3.1. Krok 1 – suma ważona (preaktywacja)

Neuron najpierw oblicza **sumę ważoną** swoich wejść $x_1, x_2, \dots, x_n$ z odpowiednimi wagami $w_1, \dots, w_n$ oraz wyrazem wolnym (biasem) b :

$$z = w_1 x_1 + w_2 x_2 + \dots + w_n x_n + b = \sum_{i=1}^n w_i x_i + b. \quad (1.3)$$

W notacji wektorowej, definiując wektory kolumnowe $\mathbf{w} = [w_1, \dots, w_n]^\top$ oraz $\mathbf{x} = [x_1, \dots, x_n]^\top$, preaktywację można zapisać zwięźle jako:

$$z = \mathbf{w}^\top \mathbf{x} + b. \quad (1.4)$$

Wartość z nazywana jest *preaktywacją* lub *logidem* neuronu. Intuicyjnie: każda waga w_i decyduje o tym, jak bardzo i -te wejście wpływa na aktywność neuronu. Bias b przesuwą próg aktywacji niezależnie od wejść.

1.3.2. Krok 2 – funkcja aktywacji

Wynik preaktywacji z jest następnie przekształcany przez **nieliniową funkcję aktywacji** ϕ , której wyjście a (ang. *activation*) jest wyjściem neuronu i wejściem dla neuronów następnej warstwy:

$$a = \varphi(z) = \varphi(\mathbf{w}^\top \mathbf{x} + b). \quad (1.5)$$

Funkcja aktywacji jest **kluczowym elementem** sieci – bez niej sieć wielowarstwowa, niezależnie od liczby warstw, realizowałaby wyłącznie transformację liniową, niezdolną do modelowania złożonych zależności.

1.4. Architektura wielowarstwowej sieci neuronowej (MLP)

Wielowarstwowy perceptron (ang. *Multilayer Perceptron*, *MLP*) składa się z warstw neuronów, w których każdy neuron jednej warstwy połączony jest ze wszystkimi neuronami warstwy poprzedniej (warstwa w pełni połączona, ang. *fully connected*). Wyróżniamy trzy typy warstw:

- **Warstwa wejściowa** (ang. *input layer*) – przyjmuje surowe dane wejściowe; nie wykonuje żadnych obliczeń, jedynie przekazuje wektor cech $\mathbf{x} \in \mathbb{R}^{d_0}$ do pierwszej warstwy ukrytej.
- **Warstwy ukryte** (ang. *hidden layers*) – realizują kolejne nieliniowe przekształcenia danych. Liczba warstw ukrytych wyznacza „głębokość” sieci.
- **Warstwa wyjściowa** (ang. *output layer*) – generuje predykcję modelu. Dla klasyfikacji K -klasowej ma K neuronów.

1.4.1. Obliczenia w warstwie – notacja macierzowa

W praktyce obliczenia prowadzi się macierzowo dla całej warstwy naraz. Niech warstwa l ma n_l neuronów, a poprzednia warstwa $l-1$ ma n_{l-1} neuronów. Definiujemy:

- $\mathbf{W}^l \in \mathbb{R}^{n_l \times n_{l-1}}$ – macierz wag warstwy l (wiersze odpowiadają neuronom, kolumny – wejściom),
- $\mathbf{b}^l \in \mathbb{R}^{n_l}$ – wektor biasów warstwy l ,
- $\mathbf{h}^{l-1} \in \mathbb{R}^{n_{l-1}}$ – wektor aktywacji poprzedniej warstwy ($\mathbf{h}^0 = \mathbf{x}$).

Propagacja w przód dla warstwy l jest wówczas:

$$\mathbf{z}^l = \mathbf{W}^l \mathbf{h}^{l-1} + \mathbf{b}^l, \quad \mathbf{h}^l = \sigma_l(\mathbf{z}^l), \quad (1.6)$$

gdzie σ_l to funkcja aktywacji warstwy l , stosowana element po elemencie do każdej składowej wektora $\mathbf{z}^l$. Wyjście ostatniej warstwy L stanowi predykcję modelu: $\hat{\mathbf{y}} = \mathbf{h}^L$.

1.5. Funkcje aktywacji

Funkcje aktywacji wprowadzają nieliniowość do modelu. Poniżej opisano najważniejsze z nich, stosowane w nowoczesnych sieciach głębokich.

1.5.1. ReLU (Rectified Linear Unit)

Funkcja ReLU [4] jest najszerzej stosowaną funkcją aktywacji w warstwach ukrytych współczesnych sieci:

$$\text{ReLU}(z) = \max(0, z). \quad (1.7)$$

Pochodna ReLU (konieczna w algorytmie backpropagation):

$$\frac{d}{dz} \text{ReLU}(z) = \begin{cases} 1 & \text{jeśli } z > 0, \\ 0 & \text{jeśli } z \leq 0. \end{cases} \quad (1.8)$$

Kluczowa zaleta: dla $z > 0$ gradient wynosi dokładnie 1, co eliminuje *problem zanikającego gradientu* (ang. *vanishing gradient problem*). Dla $z \leq 0$ neuron „umiera” (ang. *dying ReLU*). Problem ten rozwiązują warianty, np. Leaky ReLU: $\max(\alpha z, z)$, gdzie $\alpha \ll 1$.

1.5.2. Sigmoid

Historycznie pierwsza szeroko stosowana funkcja aktywacji:

$$\sigma(z) = \frac{1}{1 + e^{-z}}. \quad (1.9)$$

Zakres wyjść: $(0, 1)$. Pochodna: $\sigma'(z) = \sigma(z)(1 - \sigma(z))$. Wada: dla $|z| \gg 0$ pochodna dąży do zera, co powoduje zanikający gradient w głębokich sieciach [1].

1.5.3. Softmax – warstwa wyjściowa dla klasyfikacji wieloklasowej

Funkcja Softmax przekształca wektor K rzeczywistych liczb (logitów) $\mathbf{z} = [z_1, \dots, z_K]^\top$ w rozkład prawdopodobieństwa:

$$\text{Softmax}(\mathbf{z})_k = \frac{\exp(z_k)}{\sum_{j=1}^K \exp(z_j)}, \quad k = 1, \dots, K. \quad (1.10)$$

Dwie kluczowe właściwości gwarantowane przez Softmax:

- Wszystkie wyjścia należą do przedziału $(0, 1)$, co pozwala interpretować je jako prawdopodobieństwa: $\hat{y}_k = P(Y = k | \mathbf{x})$.
- Suma wszystkich wyjść wynosi 1: $\sum_{k=1}^K \hat{y}_k = 1$.

Przykład numeryczny dla $K = 3$ i logitów $\mathbf{z} = [2.0, 1.0, 0.5]^\top$: $\sum \exp(z_k) = 7,39 + 2,72 + 1,65 = 11,76$, stąd $\hat{\mathbf{y}} = [0,629, 0,231, 0,140]^\top$.

1.6. Funkcja straty

Funkcja straty $\mathcal{L}$ mierzy rozbieżność między predykcją sieci $\hat{\mathbf{y}}$ a prawdziwą etykietą $\mathbf{y}$.

1.6.1. Kategoryczna entropia krzyżowa

Standardową funkcją straty dla klasyfikacji K -klasowej jest **kategoryczna entropia krzyżowa** (ang. *categorical cross-entropy*), wynikająca z zasady maksymalizacji wiarygodności [1]. Dla etykiet w postaci one-hot oraz predykcji po Softmax:

$$\mathcal{L}(\mathbf{y}, \hat{\mathbf{y}}) = - \sum_{k=1}^K y_k \log \hat{y}_k. \quad (1.11)$$

Ponieważ wektor $\mathbf{y}$ jest one-hot, tylko jeden składnik sumy jest niezerowy. Wzór upraszcza się do:

$$\mathcal{L} = -\log \hat{y}_{y^*}, \quad (1.12)$$

gdzie y^* jest indeksem poprawnej klasy. Intuicja: jeśli sieć przypisuje poprawnej klasie prawdopodobieństwo $\hat{y}_{y^*} \rightarrow 1$, to $-\log \hat{y}_{y^*} \rightarrow 0$. Dla całego batchu N próbek:

$$\mathcal{L} = -\frac{1}{N} \sum_{i=1}^N \log \hat{y}_{y_i^*}. \quad (1.13)$$

1.6.2. Związek z maksymalizacją wiarygodności

Minimalizacja entropii krzyżowej jest równoważna maksymalizacji logarytmu wiarygodności modelu na danych treningowych:

$$\arg \max_{\theta} \sum_{i=1}^N P(y_i | \mathbf{x}_i; \theta) \equiv \arg \min_{\theta} - \sum_{i=1}^N \log P(y_i | \mathbf{x}_i; \theta). \quad (1.14)$$

1.7. Propagacja w przód

Propagacja w przód (ang. *forward propagation*) to deterministyczny przebieg obliczeniowy przez wszystkie warstwy sieci, od wejścia do wyjścia, w celu wygenerowania predykcji. Dla sieci L -warstwowej:

1. Inicjalizacja: $\mathbf{h}^0 = \mathbf{x}$.
2. Dla każdej warstwy $l = 1, 2, \dots, L$:

$$\mathbf{z}^l = \mathbf{W}^l \mathbf{h}^{l-1} + \mathbf{b}^l, \quad \mathbf{h}^l = \sigma_l(\mathbf{z}^l).$$

3. Predykcja: $\hat{\mathbf{y}} = \mathbf{h}^L$.

Aktywacje wszystkich warstw $\mathbf{h}^l$ i preaktywacje $\mathbf{z}^l$ są *zapamiętywane w pamięci* podczas propagacji w przód – będą potrzebne w algorytmie propagacji wstecznej [1].

1.8. Propagacja wsteczna – algorytm backpropagation

Algorytm propagacji wstecznej [2] umożliwia efektywne obliczenie gradientu funkcji straty $\mathcal{L}$ względem wszystkich parametrów sieci. Jest to zastosowanie **reguły łańcuchowej** (ang. *chain rule*) do złożenia funkcji sieci. Czas obliczeń jest liniowy względem liczby parametrów.

1.8.1. Reguła łańcuchowa

Jeśli $\mathcal{L} = \mathcal{L}(u)$ i $u = g(\mathbf{x})$, to:

$$\frac{d\mathcal{L}}{d\mathbf{x}} = \frac{d\mathcal{L}}{du} \cdot \frac{du}{d\mathbf{x}}. \quad (1.15)$$

1.8.2. Sygnał błędu warstwy wyjściowej

Definiujemy **sygnał błędu** (ang. *error signal*) warstwy l jako gradient funkcji straty względem preaktywacji:

$$\delta^l = \frac{\partial \mathcal{L}}{\partial \mathbf{z}^l}. \quad (1.16)$$

Dla warstwy wyjściowej $l = L$ z funkcjami Softmax i entropią krzyżową gradient przyjmuje elegancką postać:

$$\delta^L = \hat{\mathbf{y}} - \mathbf{y}. \quad (1.17)$$

Sygnał błędu warstwy wyjściowej to po prostu **różnica między predykcją a prawdziwą etykietą**.

1.8.3. Propagacja sygnału błędu przez warstwy ukryte

Dla dowolnej ukrytej warstwy $l < L$:

$$\delta^l = (\mathbf{W}^{l+1})^\top \delta^{l+1} \odot \sigma'_l(\mathbf{z}^l), \quad (1.18)$$

gdzie $\odot$ oznacza iloczyn element po elemencie (Hadamarda).

1.8.4. Gradienty parametrów

Mając sygnały błędu dla każdej warstwy, obliczamy gradienty wag i biasów:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}^l} = \delta^l (\mathbf{h}^{l-1})^\top, \quad \frac{\partial \mathcal{L}}{\partial \mathbf{b}^l} = \delta^l. \quad (1.19)$$

1.8.5. Pełny algorytm backpropagation

1. **Propagacja w przód:** oblicz $\mathbf{z}^l$ i $\mathbf{h}^l$ dla $l = 1, \dots, L$; zapamiętaj wszystkie wartości.
2. **Oblicz stratę** $\mathcal{L}(\hat{\mathbf{y}}, \mathbf{y})$.
3. **Oblicz** $\delta^L = \hat{\mathbf{y}} - \mathbf{y}$.
4. **Propaguj wstecz** przez warstwy $l = L-1, \dots, 1$: oblicz δ^l zgodnie z równaniem (1.18).
5. **Oblicz gradienty** $\partial \mathcal{L} / \partial \mathbf{W}^l$ i $\partial \mathcal{L} / \partial \mathbf{b}^l$ dla wszystkich l .
6. **Zaktualizuj parametry** (patrz Sekcja 1.9).

1.9. Optymalizacja parametrów – metody gradientowe

1.9.1. Podstawowy spadek gradientowy

Parametry sieci aktualizuje się w kierunku *przeciwnym do gradientu* funkcji straty:

$$\mathbf{W}^l \leftarrow \mathbf{W}^l - \eta \frac{\partial \mathcal{L}}{\partial \mathbf{W}^l}, \quad \mathbf{b}^l \leftarrow \mathbf{b}^l - \eta \frac{\partial \mathcal{L}}{\partial \mathbf{b}^l}. \quad (1.20)$$

Parametr $\eta > 0$ to **współczynnik uczenia** (ang. *learning rate*). Wyróżniamy warianty:

- **Batch GD:** gradient obliczany na całym zbiorze N próbek. Dokładny, lecz wolny.
- **SGD** (stochastic GD): gradient z jednej losowej próbki. Szybki, silnie zaszumiony.
- **Mini-batch GD:** gradient z losowego podzbioru $B \ll N$ próbek (ang. *batch size*). Najczęściej stosowany w praktyce.

1.9.2. Problem lokalnych minimów i obszarów siodłowych

Funkcja straty sieci głębokiej jest funkcją niekonweksową. Gradient może wskazywać kierunek do lokalnego minimum lub punktu siodłowego (ang. *saddle point*). W praktyce dla sieci o wielu parametrach większość lokalnych minimów ma zbliżoną wartość funkcji straty [1], a nowoczesne optymalizatory adaptacyjne radzą sobie z tym problemem skutecznie.

1.9.3. Algorytm Adam (Adaptive Moment Estimation)

Algorytm Adam [5] jest adaptacyjną metodą pierwszego rzędu, łączącą mechanizm *momentum* oraz *adaptacyjne skalowanie* współczynnika uczenia. Niech $\mathbf{g}_t$ oznacza gradient w kroku t , $\beta_1 = 0,9$, $\beta_2 = 0,999$.

Krok 1 – aktualizacja pierwszego momentu (wygładzona średnia gradientu):

$$\mathbf{m}_t = \beta_1 \mathbf{m}_{t-1} + (1 - \beta_1) \mathbf{g}_t. \quad (1.21)$$

Krok 2 – aktualizacja drugiego momentu (wygładzona wariancja gradientu):

$$\mathbf{v}_t = \beta_2 \mathbf{v}_{t-1} + (1 - \beta_2) \mathbf{g}_t^2. \quad (1.22)$$

Krok 3 – korekta biasu (ponieważ $\mathbf{m}_0 = \mathbf{v}_0 = \mathbf{0}$):

$$\hat{\mathbf{m}}_t = \frac{\mathbf{m}_t}{1 - \beta_1^t}, \quad \hat{\mathbf{v}}_t = \frac{\mathbf{v}_t}{1 - \beta_2^t}. \quad (1.23)$$

Krok 4 – aktualizacja parametrów:

$$\theta_t = \theta_{t-1} - \eta \frac{\hat{\mathbf{m}}_t}{\sqrt{\hat{\mathbf{v}}_t} + \varepsilon}, \quad (1.24)$$

gdzie stała $\varepsilon = 10^{-8}$ zapobiega dzieleniu przez zero. Adam zapewnia szybką i stabilną zbieżność przy różnorodnych architekturach i hiperparametrach.

1.10. Cykl uczenia – epoki, mini-batche i preprocessing

1.10.1. Preprocessing – standaryzacja danych

Przed podaniem danych do sieci stosuje się standaryzację (ang. *standardization*). Każdą cechę j transformuje się tak, by miała zerową średnią i jednostkowe odchylenie standardowe:

$$x'_j = \frac{x_j - \mu_j}{\sigma_j}, \quad (1.25)$$

gdzie μ_j i σ_j to odpowiednio średnia i odchylenie standardowe cechy j wyznaczone **wyłącznie na zbiorze treningowym**. Zastosowanie tych samych parametrów do zbioru testowego jest konieczne, by uniknąć wycieku danych (ang. *data leakage*).

1.10.2. Podział danych i pojęcie epoki

Zbiór danych $\mathcal{D}$ dzielony jest na trzy rozłączne podzbiory:

- **Zbiór treningowy** $\mathcal{D}_{\text{train}}$ – na nim sieć się uczy (typowo 70–80%).
- **Zbiór walidacyjny** $\mathcal{D}_{\text{val}}$ – do monitorowania uczenia i wyboru hiperparametrów (typowo 10–15%).
- **Zbiór testowy** $\mathcal{D}_{\text{test}}$ – ostateczna ocena modelu; używany tylko raz, po zakończeniu uczenia (typowo 10–20%).

Jedna **epoka** (ang. *epoch*) to jedno pełne przejście przez cały zbiór treningowy. Dane dzielone są na losowe podzbiory o rozmiarze B (ang. *mini-batch size*). Liczba kroków

optymalizacji na epokę wynosi $\lceil N/B \rceil$. Typowe wartości: $B \in \{32, 64, 128, 256\}$.

1.10.3. Ocena modelu – metryki

Podstawową metryką dla klasyfikacji jest **dokładność** (ang. *accuracy*):

$$\text{Accuracy} = \frac{1}{N} \sum_{i=1}^N \mathbf{1}[\arg \max \hat{\mathbf{y}}_i = y_i], \quad (1.26)$$

gdzie $\mathbf{1}[\cdot]$ to funkcja wskaźnikowa.

1.11. Inicjalizacja wag i regularyzacja

1.11.1. Inicjalizacja Kaiminga (He initialization)

Dobra inicjalizacja parametrów jest kluczowa. Dla warstw z aktywacją ReLU zalecana jest inicjalizacja Kaiminga [6]:

$$w^l \sim \mathcal{N}\left(0, \frac{2}{n_{l-1}}\right), \quad (1.27)$$

gdzie n_{l-1} to liczba neuronów wejściowych warstwy l (ang. *fan-in*). Czynniki 2 kompensuje fakt, że ReLU zeruje połowę aktywacji.

1.11.2. Regularyzacja – zapobieganie przetrenowaniu

Przetrenowanie (ang. *overfitting*) oznacza, że model doskonale dopasowuje się do danych treningowych, lecz słabo generalizuje na nowych danych. Główne metody regularyzacji:

- **Regularyzacja L2** (*weight decay*): dodanie członu $\frac{\lambda}{2} \sum_i \theta_i^2$ do funkcji straty. Penalizuje duże wagi, wymuszając prostsze modele.
- **Dropout** [7]: losowe „wyłączanie” neuronów z prawdopodobieństwem p podczas treningu. Zmusza sieć do uczenia redundantnych reprezentacji.
- **Early stopping**: zatrzymanie uczenia, gdy strata na zbiorze walidacyjnym przestaje maleć.

1.12. Podsumowanie – pełny cykl uczenia głębokiego

Uczenie głębokiej sieci neuronowej przebiega w następującym cyklu:

1. **Inicjalizacja**: losowe ustawienie wag $\mathbf{W}^l$ i biasów $\mathbf{b}^l$ (np. inicjalizacja Kaiminga).
2. **Preprocessing**: standaryzacja cech wejściowych.
3. **Pętla po epokach i mini-batchach**:
 - (a) *Forward pass*: oblicz predykcję $\hat{\mathbf{y}} = f(\mathbf{x}; \theta)$.
 - (b) Oblicz stratę $\mathcal{L}(\hat{\mathbf{y}}, \mathbf{y})$.

- (c) *Backward pass*: oblicz gradienty $\partial \mathcal{L} / \partial \theta$ algorytmem backpropagation.
- (d) Zaktualizuj θ optymalizatorem (Adam, SGD, ...).

4. **Ewaluacja**: oceń model na zbiorze testowym (accuracy, strata).

Dzięki backpropagation i nowoczesnym optymalizatorom możliwe jest efektywne trenowanie modeli z milionami parametrów. Automatyczne różniczkowanie (ang. *autograd*), implementowane w bibliotekach takich jak PyTorch czy TensorFlow, pozwala na budowanie dowolnych architektur bez ręcznego wyprowadzania gradientów.

2. Opis projektu

2.1. Opis zbioru danych

Sieć neuronowa uczona była na zbiorze danych *Steel Plates Faults* pochodzącym z repozytorium Uniwersytetu Kalifornijskiego w Irvine (UCI Machine Learning Repository, ID 198) [8]. Zbiór zawiera informacje o blachach stalowych i ich usterkach; zadaniem jest zbudowanie modelu, który na podstawie dostępnych cech przyporządkowuje odpowiedni typ niedoskonałości.

2.1.1. Struktura danych wejściowych

Cały zbiór danych składa się z 27 kolumn zawierających cechy (dane wejściowe), przedstawionych na Rysunku 2.1.

```
len(X.columns)
27
```

Rysunek 2.1: Nazwy kolumn – cechy zbioru danych *Steel Plates Faults*.

Szczegółowy opis poszczególnych kolumn przedstawia Rysunek 2.2.

```
for kolumna in X.columns:
    print(kolumna)

X_Minimum
X_Maximum
Y_Minimum
Y_Maximum
Pixels_Areas
X_Perimeter
Y_Perimeter
Sum_of_Luminosity
Maximum_of_Luminosity
Length_of_Conveyer
TypeOfSteel_A300
TypeOfSteel_A400
Steel_Plate_Thickness
Edges_Index
Empty_Index
Square_Index
Outside_X_Index
Edges_X_Index
Edges_Y_Index
Outside_Global_Index
LogOfAreas
Log_X_Index
Log_Y_Index
Orientation_Index
Luminosity_Index
SigmoidOfAreas
Minimum_of_Luminosity
```

Rysunek 2.2: Opis cech zbioru danych.

Zbiór nie posiada brakujących wartości. Rozkłady wartości poszczególnych cech przedstawia Rysunek 2.3. Widoczna jest znaczna rozbieżność pomiędzy poszczególnymi kolumnami, co wymusza zastosowanie normalizacji (standaryzacji) danych przed trenowaniem modelu.

```
print(df.isnull().sum().sum())
0
```

Rysunek 2.3: Rozkłady wartości cech zbioru danych – widoczna duża skala różnic między kolumnami.

```
df.iloc[:, :27].agg(['min', 'max', 'mean']).T
```

	min	max	mean
X_Minimum	0.0000	1.705000e+03	5.711360e+02
X_Maximum	4.0000	1.713000e+03	6.179645e+02
Y_Minimum	6712.0000	1.298766e+07	1.650685e+06
Y_Maximum	6724.0000	1.298769e+07	1.650739e+06
Pixels_Areas	2.0000	1.526550e+05	1.893878e+03
X_Perimeter	2.0000	1.044900e+04	1.118552e+02
Y_Perimeter	1.0000	1.815200e+04	8.296600e+01
Sum_of_Luminosity	250.0000	1.159141e+07	2.063121e+05
Maximum_of_Luminosity	37.0000	2.530000e+02	1.301937e+02
Length_of_Conveyer	1227.0000	1.794000e+03	1.459160e+03
TypeOfSteel_A300	0.0000	1.000000e+00	4.003091e-01
TypeOfSteel_A400	0.0000	1.000000e+00	5.996909e-01
Steel_Plate_Thickness	40.0000	3.000000e+02	7.873776e+01
Edges_Index	0.0000	9.952000e-01	3.317152e-01
Empty_Index	0.0000	9.439000e-01	4.142033e-01
Square_Index	0.0083	1.000000e+00	5.707671e-01
Outside_X_Index	0.0015	8.759000e-01	3.336110e-02
Edges_X_Index	0.0144	1.000000e+00	6.105286e-01
Edges_Y_Index	0.0484	1.000000e+00	8.134722e-01
Outside_Global_Index	0.0000	1.000000e+00	5.757342e-01
LogOfAreas	0.3010	5.183700e+00	2.492388e+00

Rysunek 2.4: Statystyki opisowe cech wejściowych.

Log_X_Index	0.3010	3.074100e+00	1.335686e+00
Log_Y_Index	0.0000	4.258700e+00	1.403271e+00
Orientation_Index	-0.9910	9.917000e-01	8.328764e-02
Luminosity_Index	-0.9989	6.421000e-01	-1.313050e-01
SigmoidOfAreas	0.1190	1.000000e+00	5.854205e-01
Minimum_of_Luminosity	0.0000	2.030000e+02	8.454869e+01

Rysunek 2.5: Macierz korelacji między cechami zbioru danych.

2.1.2. Zmienne docelowe

Zbiór posiada 7 etykiet/celów (danych wyjściowych), przedstawionych na Rysunku 2.6.

```
len(Y.columns)
7
```

Rysunek 2.6: Nazwy zmiennych docelowych (klas) zbioru danych.

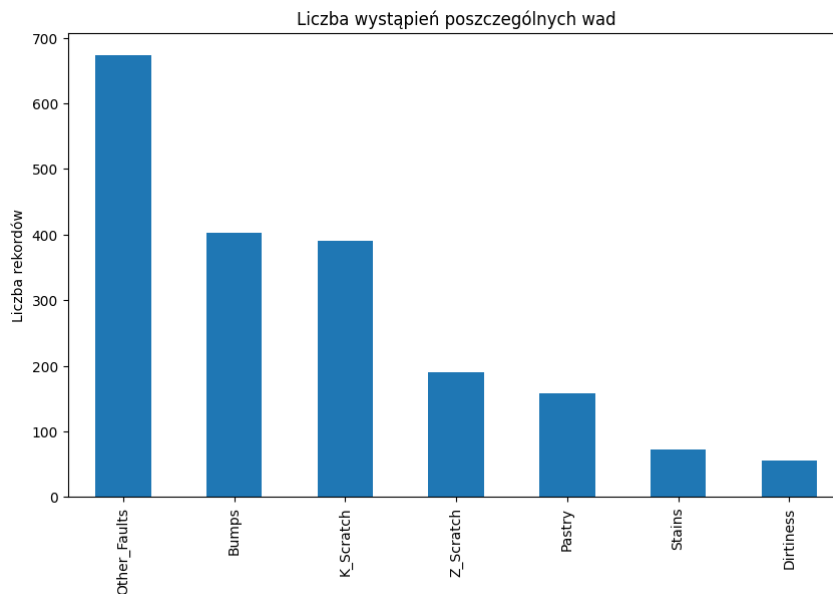
Etykiety odpowiadają następującym typom wad (Rysunek 2.7):

```
for kolumna in Y.columns:
    print(kolumna)

Pastry
Z_Scratch
K_Scratch
Stains
Dirtiness
Bumps
Other_Faults
```

Rysunek 2.7: Opis typów wad definiowanych przez zmienne docelowe.

Liczba wystąpień poszczególnych wad została przedstawiona na Rysunku 2.8. Z wykresu wynika, że najrzadszym typem wady jest `dirtyiness`, a najczęściej występującym – `other_faults`.



Rysunek 2.8: Rozkład klas – liczba wystąpień poszczególnych typów defektów.

2.2. Opis implementacji

Poniżej przedstawiono kolejne fragmenty kodu implementacji sieci neuronowej z komentarzem objaśniającym działanie każdego bloku.

2.2.1. Import bibliotek i konfiguracja akceleracji sprzętowej

W tej sekcji ładujemy niezbędne narzędzia do obliczeń (PyTorch), przetwarzania danych (NumPy, Scikit-learn) oraz sprawdzamy, czy model może być trenowany na karcie graficznej (CUDA).

```
1 import torch
2 import torch.nn as nn
3 import torch.nn.functional as F
4 import torch.optim as optim
5 from torch.utils.data import DataLoader, TensorDataset
6 from sklearn.model_selection import train_test_split
7 from sklearn.preprocessing import StandardScaler
8 import numpy as np
9
10 device = "cuda" if torch.cuda.is_available() else "cpu"
```

Listing 2.1: Import bibliotek i wykrywanie urządzenia obliczeniowego.

2.2.2. Pobranie danych i rozdzielenie cech od etykiet

Pobieramy zestaw danych o defektach stali za pomocą dedykowanego identyfikatora UCI i przypisujemy dane wejściowe (cechy) do zmiennej `X` oraz etykiety do zmiennej `y`.

```
1 from ucimlrepo import fetch_ucirepo
2 steel_plates_faults = fetch_ucirepo(id=198)
3
4 X = steel_plates_faults.data.features
5 y = steel_plates_faults.data.targets
```

Listing 2.2: Pobranie zbioru danych Steel Plates Faults z UCI.

2.2.3. Konwersja etykiet z formatu one-hot na indeksy klas

PyTorch w klasyfikacji wieloklasowej oczekuje numeru klasy (0, 1, 2, ...), a nie macierzy zer i jedynek, dlatego używamy funkcji `argmax`.

```
1 y = np.argmax(y.values, axis=1)
```

Listing 2.3: Konwersja etykiet one-hot na indeksy klas.

2.2.4. Podział danych na zbiór treningowy i testowy

Dane dzielimy w stosunku 80/20. Parametr `stratify=y` gwarantuje, że w obu zbiorach będziemy mieli taki sam procentowy udział każdego z 7 typów defektów.

```
1 X_train, X_test, y_train, y_test = train_test_split(
2     X, y, test_size=0.2, random_state=42, stratify=y
3 )
```

Listing 2.4: Podział danych na podzbiory treningowy i testowy (80/20).

2.2.5. Standaryzacja danych wejściowych

Skalujemy dane tak, aby każda cecha miała średnią 0 i odchylenie standardowe 1 (patrz równanie (1.25)).

```
1 scaler = StandardScaler()
2 X_train_scaled = scaler.fit_transform(X_train)
3 X_test_scaled = scaler.transform(X_test)
```

Listing 2.5: Standaryzacja cech za pomocą `StandardScaler`.

2.2.6. Konwersja danych na tensory PyTorch

Zamieniamy tablice NumPy na format akceptowany przez sieć neuronową i przesyłamy je na GPU lub CPU.

```
1 X_train_tensor =  
    torch.from_numpy(X_train_scaled).float().to(device)  
2 X_test_tensor =  
    torch.from_numpy(X_test_scaled).float().to(device)  
3 y_train_tensor = torch.from_numpy(y_train).long().to(device)  
4 y_test_tensor = torch.from_numpy(y_test).long().to(device)
```

Listing 2.6: Konwersja danych na tensory i przesłanie na urządzenie.

2.2.7. Utworzenie iteratorów danych (DataLoader)

Łączymy cechy z etykietami w jeden obiekt i tworzymy loader, który będzie podawał sieci dane w paczkach po 32 rekordy, mieszając je przed każdą epoką.

```
1 train_dataset = TensorDataset(X_train_tensor, y_train_tensor)  
2 train_loader = DataLoader(train_dataset, batch_size=32,  
    shuffle=True)
```

Listing 2.7: Tworzenie obiektu DataLoader do iteracji po mini-batchach.

2.2.8. Definicja architektury sieci neuronowej

Tworzymy klasę modelu z dwiema warstwami ukrytymi (128 i 64 neurony). Metoda forward definiuje przepływ danych przez aktywacje ReLU.

```
1 class SteelFaultNet(nn.Module):  
2     def __init__(self):  
3         super(SteelFaultNet, self).__init__()  
4         self.fc1 = nn.Linear(27, 128)    # 27 cech wejściowych  
5         self.fc2 = nn.Linear(128, 64)  
6         self.fc3 = nn.Linear(64, 7)      # 7 klas  
7  
8     def forward(self, x):  
9         x = F.relu(self.fc1(x))  
10        x = F.relu(self.fc2(x))  
11        x = self.fc3(x)  
12        return x  
13  
14 model = SteelFaultNet().to(device)
```

Listing 2.8: Definicja klasy modelu – wielowarstwowy perceptron MLP.

Uwaga: W wersji eksperymentalnej kodu stosowano warstwę Dropout (`nn.Dropout(0.2)`) jako regularyzację, jednak warstwa ta była umieszczona nieprawidłowo – po obliczeniu

wyniku ostatniej warstwy, zamiast przed nią. W ostatecznej wersji modelu Dropout nie jest stosowany.

2.2.9. Konfiguracja funkcji straty i optymalizatora

Definiujemy `CrossEntropyLoss` jako miernik błędu oraz Adam jako algorytm optymalizujący wagi sieci.

```
1 criterion = nn.CrossEntropyLoss()
2 optimizer = optim.Adam(model.parameters(), lr=0.001)
```

Listing 2.9: Konfiguracja funkcji straty i optymalizatora Adam.

2.2.10. Pętla treningowa modelu

Przechodzimy przez dane 20 razy (epoki). Dla każdej paczki danych zerujemy gradienty, obliczamy wynik, sprawdzamy błąd (loss), wykonujemy propagację wsteczną i aktualizujemy wagi.

```
1 epochs = 20
2
3 for epoch in range(epochs):
4     model.train()
5     running_loss = 0.0
6     correct = 0
7     total = 0
8
9     for x_batch, y_batch in train_loader:
10         optimizer.zero_grad()
11         outputs = model(x_batch)
12         loss = criterion(outputs, y_batch)
13         loss.backward()
14         optimizer.step()
15
16         running_loss += loss.item()
17         _, predicted = torch.max(outputs, 1)
18         total += y_batch.size(0)
19         correct += (predicted == y_batch).sum().item()
20
21     avg_loss = running_loss / len(train_loader)
22     train_acc = 100 * correct / total
23     print(f"Epoch {epoch+1:2d} | Loss: {avg_loss:.4f} "
24           f"| Train Acc: {train_acc:.2f}%")
```

Listing 2.10: Pętla treningowa – 20 epok z mini-batchami.

2.2.11. Ewaluacja końcowa na zbiorze testowym

Wyłączamy obliczanie gradientów i sprawdzamy, jak model radzi sobie z danymi, których nigdy wcześniej nie widział.

```
1 with torch.no_grad():
2     model.eval()
3     outputs = model(X_test_tensor)
4     loss     = criterion(outputs, y_test_tensor).item()
5     _, predicted = torch.max(outputs, 1)
6     accuracy = (predicted ==
7                 y_test_tensor).float().mean().item() * 100
8
9 print(f"\nTest Loss:      {loss:.4f}")
10 print(f"Test Accuracy: {accuracy:.2f}%")
```

Listing 2.11: Ewaluacja końcowa na zbiorze testowym.

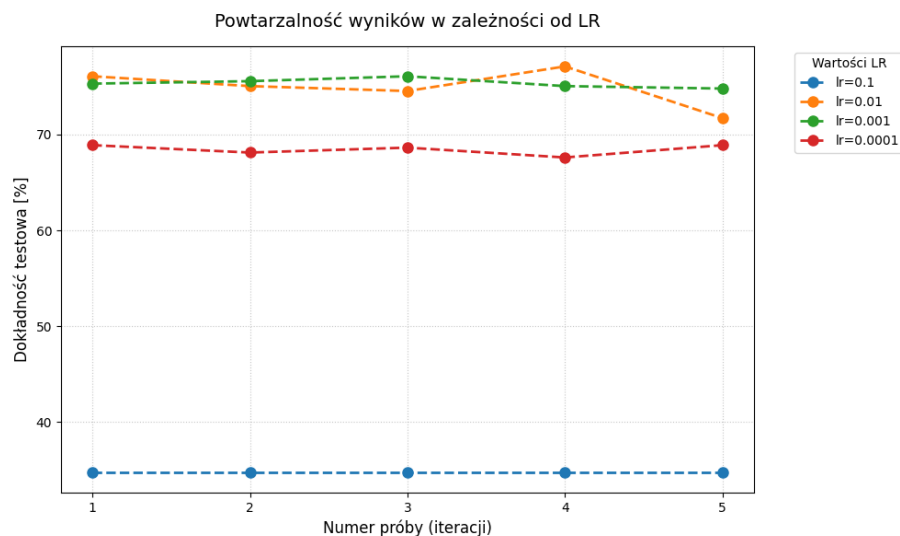
3. Eksperymenty

Wszystkie eksperymenty przeprowadzono na architekturze bazowej ($27 \rightarrow 128 \rightarrow 64 \rightarrow 7$) z optymalizatorem Adam przez 20 epok. Każda konfiguracja była testowana w 3 niezależnych iteracjach w celu oceny stabilności wyników.

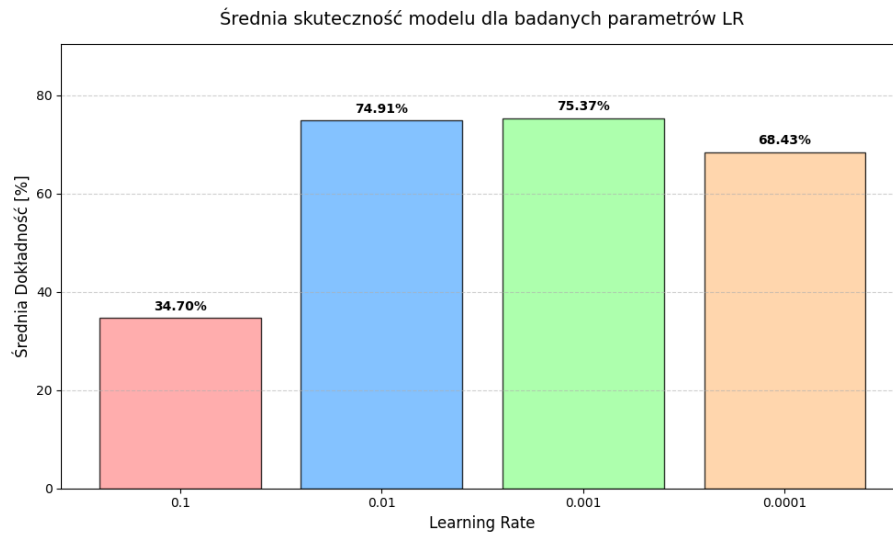
3.1. Wpływ współczynnika uczenia

Pierwszym badanym hiperparametrem był współczynnik uczenia (*learning rate*). Parametr ten definiuje szybkość aktualizacji wag modelu – jego właściwy dobór jest krytyczny dla uzyskania stabilnej zbieżności sieci oraz wysokiej precyzji klasyfikacji.

Badaniu poddano cztery wartości: $\eta \in \{0,1; 0,01; 0,001; 0,0001\}$.



Rysunek 3.1: Powtarzalność wyników skuteczności modelu w zależności od współczynnika uczenia – 3 niezależne przebiegi dla każdej wartości η .



Rysunek 3.2: Średnia skuteczność klasyfikacji na zbiorze testowym dla badanych wartości współczynnika uczenia.

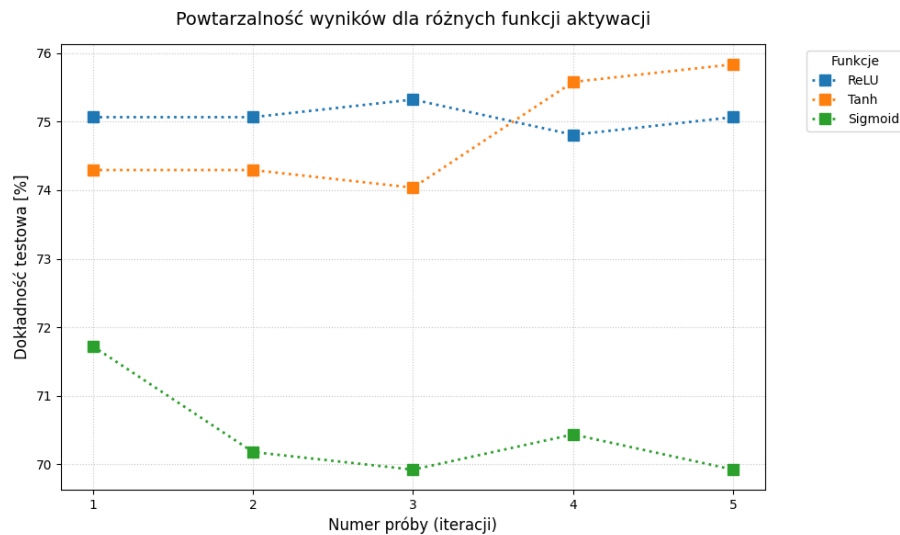
Z wykresów (Rysunki 3.1–3.2) widać, że najlepsze wyniki osiągnięto dla $\eta = 0,001$, natomiast najgorsza zbieżność wystąpiła dla $\eta = 0,1$ (niestabilne uczenie z oscylacjami funkcji straty).

3.2. Porównanie funkcji aktywacji

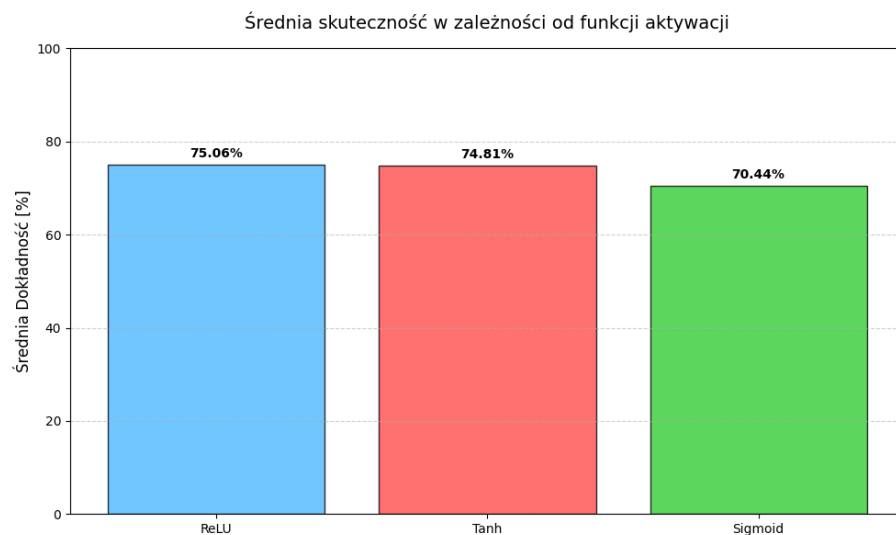
Celem drugiego eksperymentu było zbadanie wpływu różnych funkcji aktywacji na proces uczenia i finalną precyzję modelu. Przetestowano trzy funkcje:

- **ReLU** (*Rectified Linear Unit*) – standard eliminujący problem zanikającego gradientu,
- **Tanh** (tangens hiperboliczny) – skaluje wyjścia do zakresu $(-1, 1)$,
- **Sigmoid** – klasyczna funkcja logistyczna skalująca wyjścia do $(0, 1)$.

We wszystkich próbach zastosowano stały współczynnik uczenia $\eta = 0,001$.



Rysunek 3.3: Powtarzalność wyników dla różnych funkcji aktywacji – 3 niezależne przebiegi dla każdej funkcji.



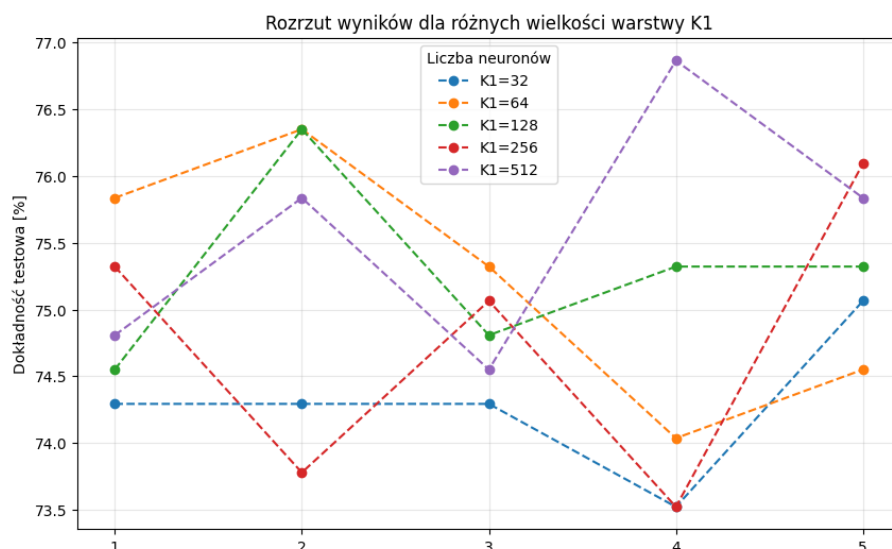
Rysunek 3.4: Średnia skuteczność klasyfikacji w zależności od zastosowanej funkcji aktywacji.

Przeprowadzony eksperyment wykazał, że funkcja **ReLU** jest najbardziej efektywnym rozwiązaniem dla założonej architektury sieci neuronowej (Rysunki 3.3 i 3.4).

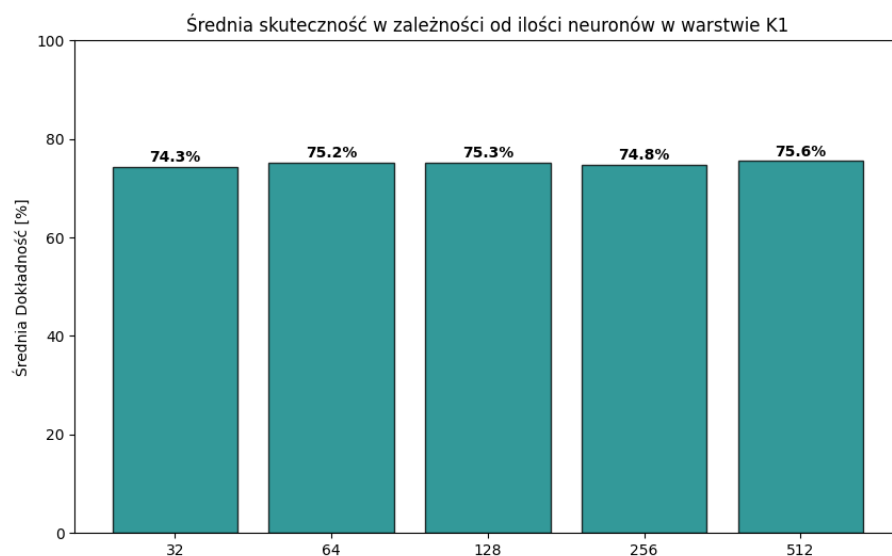
3.3. Wpływ liczby neuronów w pierwszej warstwie ukrytej (K_1)

Celem eksperymentu była analiza wpływu liczby neuronów w pierwszej warstwie ukrytej (K_1) na zdolność modelu do wstępnego przetwarzania 27 cech wejściowych.

Przyjęto stałą wartość drugiej warstwy $K_2 = 64$ oraz stałe parametry: $\eta = 0,001$, aktywacja ReLU. Testowano wartości: $K_1 \in \{32, 64, 128, 256, 512\}$.



Rysunek 3.5: Rozrzut wyników skuteczności dla różnych rozmiarów warstwy K_1 .



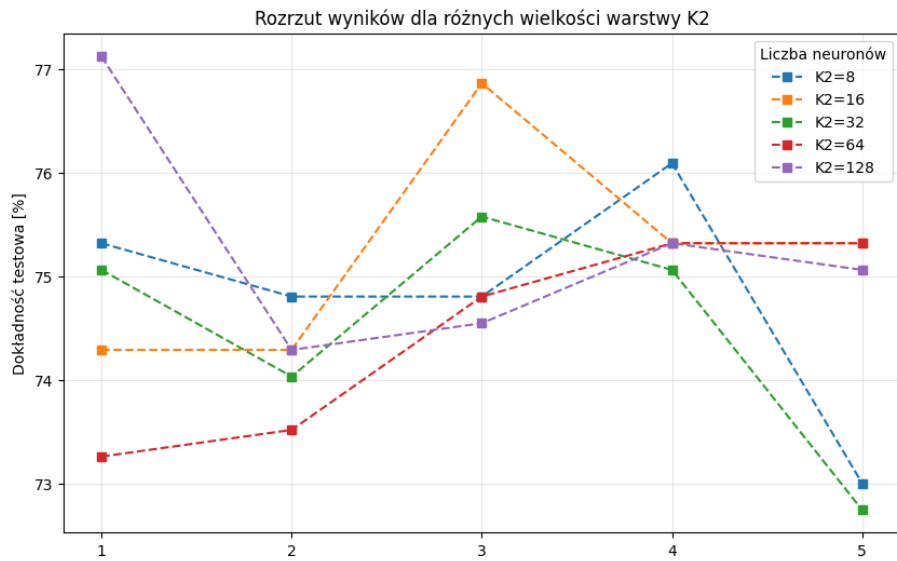
Rysunek 3.6: Średnia skuteczność klasyfikacji w zależności od liczby neuronów w warstwie K_1 .

Najlepsze wyniki uzyskano dla $K_1 = 512$ (Rysunki 3.5 i 3.6).

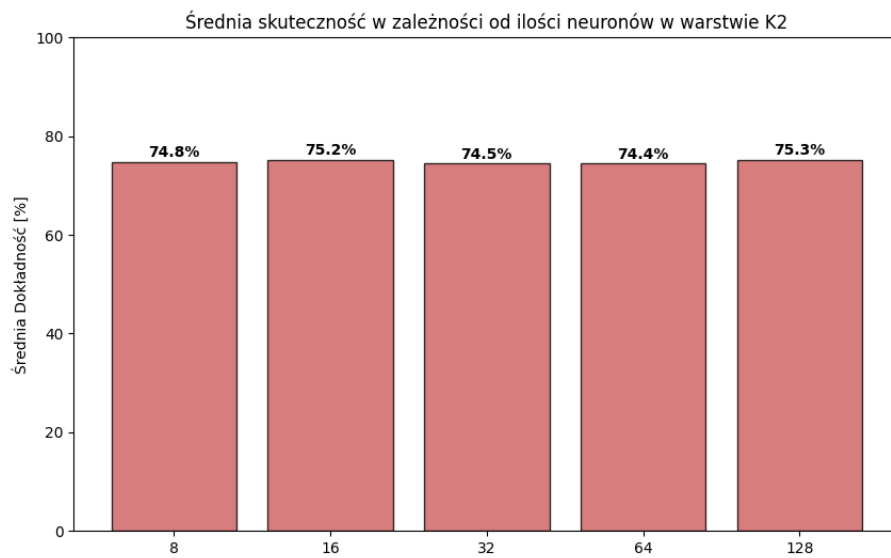
3.4. Wpływ liczby neuronów w drugiej warstwie ukrytej (K_2)

Eksperyment miał na celu sprawdzenie, jak liczba neuronów w drugiej warstwie ukrytej (K_2) wpływa na ostateczną decyzję klasyfikatora.

Przyjęto stałą wartość pierwszej warstwy $K_1 = 128$ oraz stałe parametry: $\eta = 0,001$, aktywacja ReLU. Testowano wartości: $K_2 \in \{8, 16, 32, 64, 128\}$.



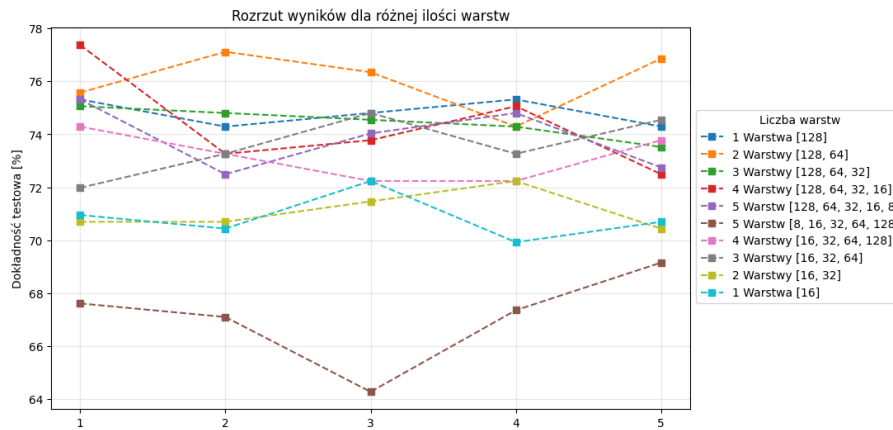
Rysunek 3.7: Rozrzut wyników skuteczności dla różnych rozmiarów warstwy K_2 .



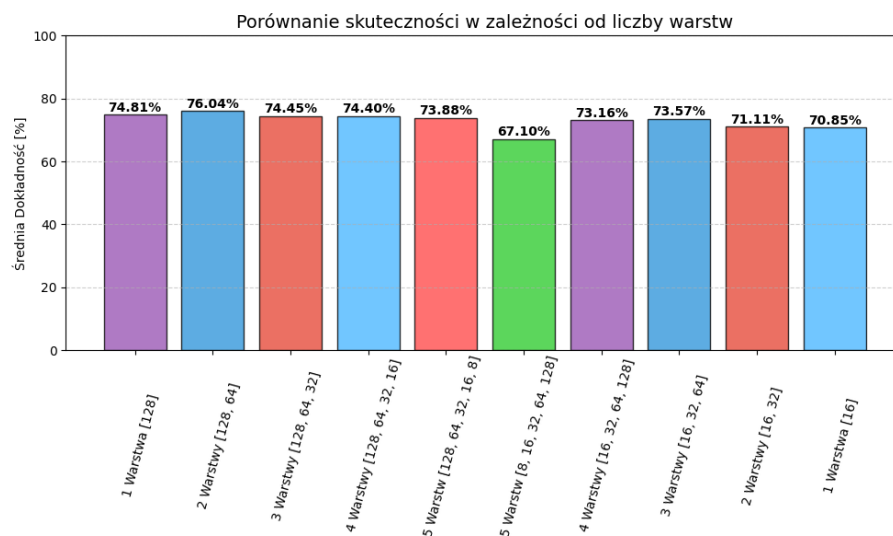
Rysunek 3.8: Średnia skuteczność klasyfikacji w zależności od liczby neuronów w warstwie K_2 .

3.5. Wpływ liczby warstw ukrytych

Zbadano wpływ głębokości modelu na precyzję klasyfikacji, testując sieci z różną liczbą warstw ukrytych.



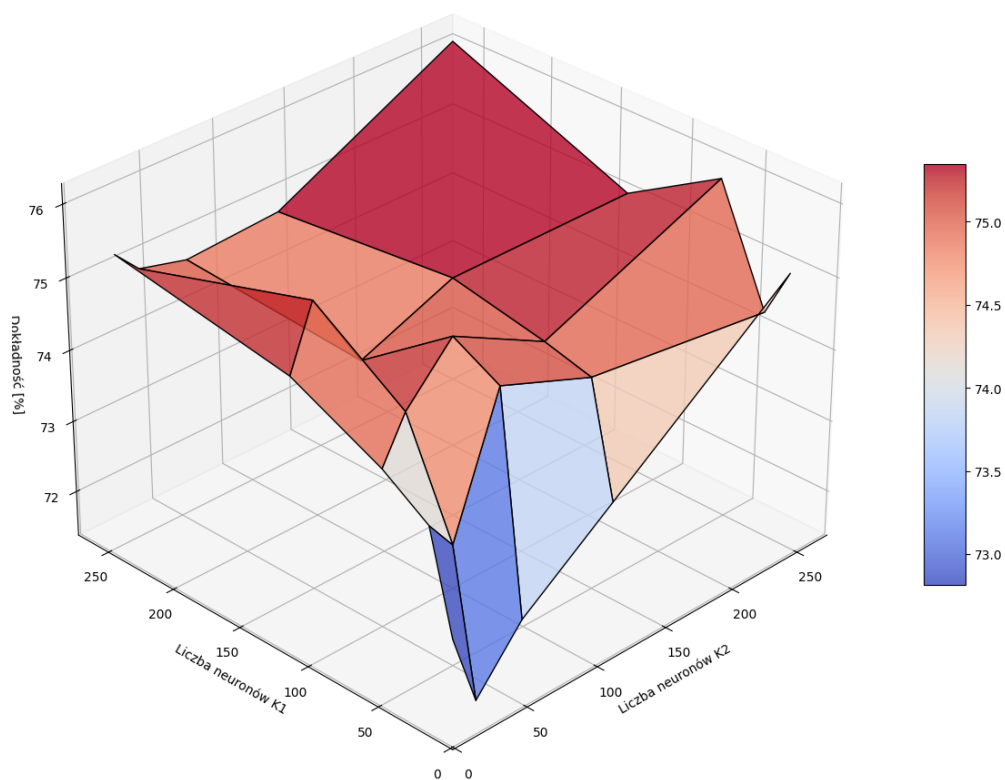
Rysunek 3.9: Rozrzut wyników skuteczności dla różnych liczb warstw ukrytych.



Rysunek 3.10: Średnia skuteczność klasyfikacji w zależności od liczby warstw ukrytych.

3.6. Wspólny wpływ liczby neuronów K_1 i K_2

Ostatni eksperyment skupiał się na znalezieniu optymalnej relacji między pierwszą (K_1) a drugą (K_2) warstwą ukrytą – przeszukiwano siatka kombinacji obu parametrów.

Wpływ ilości neuronów K_1 i K_2 na skuteczność modelu**Rysunek 3.11:** Mapa skuteczności klasyfikacji w zależności od kombinacji K_1 i K_2 .

4. Podsumowanie

W ramach niniejszego projektu zrealizowano proces projektowania, implementacji oraz wieloaspektowej optymalizacji głębokiej sieci neuronowej typu MLP (*Multilayer Perceptron*) przeznaczonej do klasyfikacji defektów blach stalowych. Na podstawie przeprowadzonych serii eksperymentów sformułowano następujące wnioski końcowe:

1. **Dobór parametrów uczących.** Eksperymentalnie wykazano, że dla badanego zbioru danych optymalny balans między szybkością zbieżności a stabilnością wyników zapewnia współczynnik uczenia $\eta = 0,001$. Zastosowanie funkcji aktywacji **ReLU** pozwoliło na najskuteczniejszą eliminację problemu zanikającego gradientu, co przełożyło się na wysoką precyzję klasyfikacji w porównaniu do klasycznych funkcji Sigmoid i Tanh.
2. **Analiza architektury warstw ukrytych.** Badania nad strukturą sieci (szerokość K_1 i K_2) oraz jej głębokością pozwoliły zidentyfikować punkt nasycenia modelu. Ustalono, że trójwarstwowa architektura o konfiguracji [256, 256] jest wariantem optymalnym.
3. **Skuteczność klasyfikacji.** Finalny model uzyskał średnią skuteczność na poziomie **76,18%**. Biorąc pod uwagę fakt, że problem dotyczył klasyfikacji wieloklasowej (7 różnych typów defektów), wynik ten potwierdza wysoką przydatność głębokich sieci neuronowych w automatyzacji systemów kontroli jakości w przemyśle metalurgicznym.

Przeprowadzone prace badawcze pozwoliły nie tylko na stworzenie skutecznego klasyfikatora, ale również na dogłębne zrozumienie zależności zachodzących pomiędzy hiperparametrami sieci a charakterystyką procesu uczenia dla danych tabelarycznych.

Literatura

- [1] Ian Goodfellow, Yoshua Bengio i Aaron Courville. *Deep Learning*. Cambridge, MA: MIT Press, 2016. URL: <https://www.deeplearningbook.org>.
- [2] David E. Rumelhart, Geoffrey E. Hinton i Ronald J. Williams. „Learning representations by back-propagating errors”. W: *Nature* 323 (1986), s. 533–536. DOI: [10.1038/323533a0](https://doi.org/10.1038/323533a0).
- [3] Alex Krizhevsky, Ilya Sutskever i Geoffrey E. Hinton. „ImageNet Classification with Deep Convolutional Neural Networks”. W: *Advances in Neural Information Processing Systems (NIPS 2012)*. T. 25. 2012.
- [4] Vinod Nair i Geoffrey E. Hinton. „Rectified Linear Units Improve Restricted Boltzmann Machines”. W: *Proceedings of the 27th International Conference on Machine Learning (ICML 2010)*. 2010, s. 807–814.
- [5] Diederik P. Kingma i Jimmy Ba. „Adam: A Method for Stochastic Optimization”. W: *Proceedings of the 3rd International Conference on Learning Representations (ICLR 2015)*. 2015. arXiv: [1412.6980](https://arxiv.org/abs/1412.6980).
- [6] Kaiming He, Xiangyu Zhang, Shaoqing Ren i Jian Sun. „Delving Deep into Rectifiers: Surpassing Human-Level Performance on ImageNet Classification”. W: *Proceedings of the IEEE International Conference on Computer Vision (ICCV 2015)*. 2015, s. 1026–1034. DOI: [10.1109/ICCV.2015.123](https://doi.org/10.1109/ICCV.2015.123).
- [7] Nitish Srivastava, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever i Ruslan Salakhutdinov. „Dropout: A Simple Way to Prevent Neural Networks from Overfitting”. W: *Journal of Machine Learning Research* 15.56 (2014), s. 1929–1958.
- [8] UCI Machine Learning Repository. *Steel Plates Faults Dataset (ID 198)*. Dostęp: 2025. 2010. URL: <https://archive.ics.uci.edu/dataset/198/steel+plates+faults>.

Spis rysunków

2.1	Nazwy kolumn – cechy zbioru danych <i>Steel Plates Faults</i>	14
2.2	Opis cech zbioru danych.	14
2.3	Rozkłady wartości cech zbioru danych – widoczna duża skala różnic między kolumnami.	15
2.4	Statystyki opisowe cech wejściowych.	15
2.5	Macierz korelacji między cechami zbioru danych.	15
2.6	Nazwy zmiennych docelowych (klas) zbioru danych.	15
2.7	Opis typów wad definiowanych przez zmienne docelowe.	15
2.8	Rozkład klas – liczba wystąpień poszczególnych typów defektów.	16
3.1	Powtarzalność wyników skuteczności modelu w zależności od współczynnika uczenia – 3 niezależne przebiegi dla każdej wartości η	21
3.2	Średnia skuteczność klasyfikacji na zbiorze testowym dla badanych wartości współczynnika uczenia.	22
3.3	Powtarzalność wyników dla różnych funkcji aktywacji – 3 niezależne przebiegi dla każdej funkcji.	23
3.4	Średnia skuteczność klasyfikacji w zależności od zastosowanej funkcji aktywacji.	23
3.5	Rozrzut wyników skuteczności dla różnych rozmiarów warstwy K_1	24
3.6	Średnia skuteczność klasyfikacji w zależności od liczby neuronów w warstwie K_1	24
3.7	Rozrzut wyników skuteczności dla różnych rozmiarów warstwy K_2	25
3.8	Średnia skuteczność klasyfikacji w zależności od liczby neuronów w warstwie K_2	25
3.9	Rozrzut wyników skuteczności dla różnych liczb warstw ukrytych.	26
3.10	Średnia skuteczność klasyfikacji w zależności od liczby warstw ukrytych.	26
3.11	Mapa skuteczności klasyfikacji w zależności od kombinacji K_1 i K_2	27

Listings

2.1	Import bibliotek i wykrywanie urządzenia obliczeniowego.	16
2.2	Pobranie zbioru danych Steel Plates Faults z UCI.	17
2.3	Konwersja etykiet one-hot na indeksy klas.	17
2.4	Podział danych na podzbiory treningowy i testowy (80/20).	17
2.5	Standaryzacja cech za pomocą StandardScaler.	17
2.6	Konwersja danych na tensory i przesłanie na urządzenie.	18
2.7	Tworzenie obiektu DataLoader do iteracji po mini-batchach.	18
2.8	Definicja klasy modelu – wielowarstwowy perceptron MLP.	18
2.9	Konfiguracja funkcji straty i optymalizatora Adam.	19
2.10	Pętla treningowa – 20 epok z mini-batchami.	19
2.11	Ewaluacja końcowa na zbiorze testowym.	20